
pixi-sbom: Bill-of-Materials Generation for Mixed-Ecosystem Environments

Design, identity resolution and evaluation

Kevin Mills

September 2026

ABSTRACT

pixi-sbom generates CycloneDX and SPDX documents from pixi and conda environments. This paper describes its design, with emphasis on the problem that motivated it: conda environments mix package ecosystems, and a component identified by the ecosystem it was installed from is frequently not identifiable by the ecosystem a consumer can query. We describe the intermediate model that decouples input sources from output formats, the dual-identity scheme that lets one component carry both its provenance and its queryable identity, the enrichment pipeline and its caching strategy, and the interface stability contract introduced at 1.0 and enforced by tests that read the specification document. We evaluate the tool on 118 public workspaces containing 15,722 packages, where identity resolution surfaced 1,274 advisories that conventional identification did not, without losing any.

Introduction

A bill of materials generator for a single-ecosystem project is a straightforward program: read the lockfile, emit the components. The problems that make pixi-sbom interesting all come from conda environments not being single-ecosystem.

A conda environment resolves Python packages, C and C++ shared libraries, compilers, CUDA runtimes, R packages and system tooling together, through one solver, from channels that serve all of them in the same package format. pixi adds a workspace model on top: named environments, per-platform resolution, and a lockfile that records every environment and platform in one file.

Three consequences shape the design.

Identity is ambiguous. A Python package installed from conda-forge has a conda identity and a PyPI identity. Both are true. They are useful for different things, and no single-identifier scheme serves both.

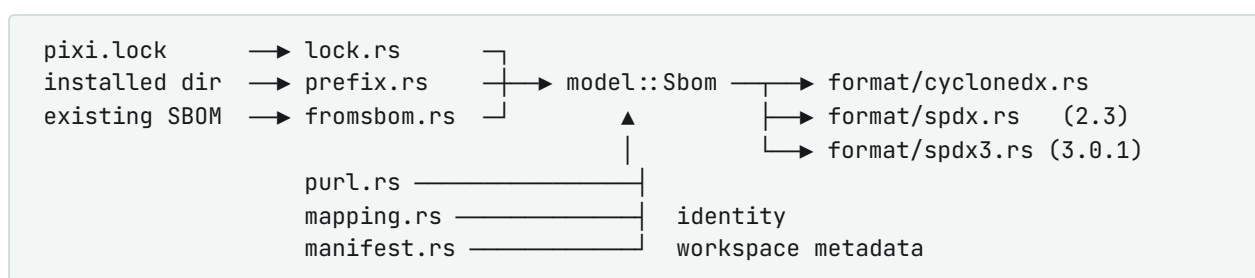
The lockfile is the artifact. In continuous integration and in supply chain review, the environment usually has not been built. A tool that requires a materialized environment cannot answer the question at the point it is asked.

One lockfile is many documents. A pixi lockfile may describe several environments across several platforms. `default` on `linux-64` and `test` on `osx-arm64` are different component sets and warrant different documents.

This paper describes how `pixi-sbom` handles these, what it deliberately does not do, and how it is tested.

Architecture

The program is a single Rust binary, roughly 18,500 lines of non-test source, structured as a short pipeline that meets at a format-agnostic intermediate model.



`model.rs` defines `Sbom`, `Root`, `Package` and `PackageKind` as plain data, with no serde derives and no knowledge of any specification. This is the central design decision and everything else follows from it.

The benefit is not abstraction for its own sake. It is that lockfile interpretation happens exactly once regardless of how many output formats exist, and conversely that a new input source, such as reading an installed prefix or ingesting a third-party document, reaches every format for free. When SPDX 3.0.1 support was added, no lockfile code changed. When `--prefix` was added, no format code changed.

Input sources

`lock.rs` parses the lockfile through `rattler_lock`, the same library the package managers use, selects an environment and platform, and resolves the dependency graph. All lockfile-shape knowledge is confined here. Delegating the parse rather than hand-rolling it means new lockfile versions are somebody else's problem, which has repeatedly turned out to be the right trade.

`prefix.rs` reads an installed environment instead: `conda-meta/*.json` records for conda packages, `site-packages/*.dist-info` for pip-installed ones. It consults each distribution's `INSTALLER` so that a package conda installed is not also counted as a PyPI package. This path serves containers and `pixi global` environments, where no lockfile is present.

`fromsbom.rs` ingests an existing CycloneDX or SPDX document into the same model, which lets the reports, the license policy and the vulnerability gate run against documents this tool

did not produce.

Output formats

Four specification targets share one mapping problem and are implemented independently because their data models genuinely differ.

CycloneDX 1.6 and 1.7 are close relatives; 1.7 adds a citation attributing the inventory to its source. SPDX 2.3 is a flat document with `SPDXRef` identifiers and `LicenseRef` extraction. SPDX 3.0.1 is a JSON-LD graph in which each element class is a node, requiring minted IRIs, deduplicated license and supplier elements, and explicit `dependsOn` and `hasDeclaredLicense` relationships.

SPDX 3.0.1 support includes the security profile, so vulnerability findings become `security_Vulnerability` elements with `security_CvssV3VulnAssessmentRelationship`, `security_ExploitCatalogVulnAssessmentRelationship` for KEV entries, and `security_VexNotAffectedVulnAssessmentRelationship` where a finding has been assessed.

One implementation note worth recording, because it cost time: SPDX 3.0.1's schema rejects timestamps with sub-second precision. OSV returns nanosecond timestamps. Documents validated locally and failed against the published schema until timestamps were normalized to whole seconds.

Identity resolution

This is the part of the design that exists because of the ecosystem problem, and it is the reason the tool was written.

The problem

A conda-forge package named `pillow` is the Python imaging library, distributed as a conda package. Its natural identifier is `pkg:conda/pillow@10.2.0`. That identifier is accurate, and no vulnerability database indexes the `pkg:conda` namespace, so a scanner reading it returns nothing. Not an error. An empty list.

The conda identity is not worthless, which is why the obvious fix of emitting PyPI identifiers instead is wrong. It records the channel, the build string and the platform. It answers reproducibility and channel-trust questions the PyPI identifier cannot answer, because `pkg:pypi/pillow@10.2.0` does not distinguish a conda-forge build from a wheel from PyPI, and those are different artifacts with different build provenance.

Both identities are true and each is load-bearing for a different consumer.

The scheme

pixi-sbom gives a component both, and lets the operator choose which one is offered for matching.

Identities come from two sources. Modern pixi lockfiles record a `urls:` field for conda packages whose PyPI identity the solver knew; `mapping.rs` uses these first because they are authoritative for that lockfile. For the rest it consults the conda-forge conda-to-PyPI name mapping, the same mapping the package managers consume, downloaded and cached for a day, or supplied offline as a file for air-gapped use.

`--primary-purl` then selects which identity occupies the component's `purl` field, with the other retained as a secondary reference:

SETTING	PURL	SECONDARY	SERVES
conda (default)	pkg:conda/ ...	pkg:pypi/ ...	provenance, reproducibility
pypi	pkg:pypi/ ...	pkg:conda/ ...	vulnerability and license matching

The default is `conda`, on the principle that a tool should describe what is actually installed unless told otherwise. This is arguably the wrong default for the most common use, and it is retained because changing it after 1.0 would break the interface contract described below.

Two restraints matter as much as the mechanism.

No guessing. In the evaluation corpus, 67.6 percent of conda packages had no PyPI equivalent, because they are compilers, shared libraries and non-Python packages. A tool that inferred identities from name similarity would produce false matches for these. False matches are worse than absent ones, because they consume triage effort and erode trust in every other finding. Only the published mapping and the lockfile's own record are consulted.

Nothing is discarded. Because the secondary identity is retained, resolution is purely additive. The evaluation confirms this empirically: across 118 workspaces, no advisory found under conda identification was absent under PyPI identification.

Enrichment

Optional passes fill fields the lockfile does not carry. Each is off by default, because each costs network time and the base document should be fast and offline.

Licenses (`--fetch-licenses`) resolve through a deliberate fallback chain that prefers local sources: the rattler package cache on disk (`pkgcache.rs`), then the channel archive over HTTP range requests (`condaarchive.rs`), then the wheel's own `dist-info` (`wheel.rs`), then the PyPI

index JSON (`pypi.rs`). The same pass collects PEP 592 yanked status and the repository URLs that `--scorecard` later needs.

Vulnerabilities (`--vulnerabilities osv`) collect every queryable purl, excluding conda purls because they cannot be queried, batch them to OSV’s `querybatch` endpoint, fetch each record in parallel, merge GHSA and PYSEC records that are aliases of one another, and select the fixed version above the installed one. A failed batch query is fatal; a failed individual record is not, on the reasoning that a partial advisory set silently presented as complete is the exact failure this tool exists to prevent.

KEV (`--kev`) cross-references CISA’s Known Exploited Vulnerabilities catalog by CVE alias and rates matches critical with the catalog’s remediation dates.

Scorecard (`--scorecard`) fetches OpenSSF Scorecard results per repository.

Reports (`--report`) render the model as tables rather than documents: `packages` , `licenses` , `outdated` , `scorecard` , `python` , `phantom` and `diff` . `phantom` is the most unusual: `imports.rs` reads the workspace’s own `.py` files for top-level imports without executing Python or using a parser crate, and `phantom.rs` compares them against the declared and installed sets to find imports satisfied by a transitive dependency rather than a declared one.

Concurrency and caching

`concurrency.rs` decides parallelism once for the whole run: one thread per core, never more than ten requests in flight, overridable through `PIXI_SBOM_CONCURRENCY` . Network jobs run on a dedicated rayon pool so a slow upstream cannot starve the rest of the program, and results are returned in job order however the threads interleave.

Caches have lifetimes matched to how fast their source changes:

CACHE	LIFETIME
conda-to-PyPI mapping	1 day
OSV queries	1 hour
OSV records	until the record’s <code>modified</code> advances
KEV catalog	1 day
Scorecard	1 week
Wheel and conda metadata	content-addressed

`batch.rs` handles a run that writes many documents, which is the common case for `--all-environments` . Rather than looking packages up once per document, it takes the union across every document and performs one pass. What that pass learned is determined by diffing package state before and after, so a per-environment fact such as `pixi:direct` or a

dependency edge is never copied across documents, and an enrichment step added later is carried without editing `batch.rs` .

Interface stability

Version 1.0 froze the command-line interface, and `docs/stability.md` specifies exactly what that covers: the flags, the exit codes, the configuration keys, the environment variables, the `pixi:*` property names that appear in documents, the report JSON shape, and the GitHub Action's inputs and outputs.

The mechanism that makes this more than a promise is `tests/stability.rs` , which checks the document against the code **in both directions**. A flag in the code and not in the document fails. A flag in the document and not in the code fails. The same holds for the other frozen surfaces.

Two details of that test are worth recording because they came from real failures.

The configuration keys are not compared against the Rust struct field names. Because the deserializer uses `rename_all = "kebab-case"` with `deny_unknown_fields` , the field names and the accepted keys differ, and an earlier version of the test compared the document against the field names and passed while every documented key was in fact rejected by the parser. The test now derives the accepted keys from the parser's own rejection message, which cannot drift from the parser's behavior because it is produced by it.

The test also normalizes line endings before parsing the action definition. A CRLF checkout on Windows broke a string search for `"\ninputs:\n"` and the failure was reported only on that platform.

Two renames landed at 1.0, `--name` to `--root-name` and `--outdated-only` to `--outdated-min` . Both old spellings are permanent hidden aliases rather than deprecations on a timer. Permanent aliases cost one line each and mean no existing invocation ever breaks, which removed the need for a deprecation release before 1.0.

Exit codes are frozen at `0 1 2 3 4 6 7 8 9` . **5 is deliberately unused** and stays unassigned, because it has never been assigned in any release and assigning it now would give a number meaning where a script might reasonably have assumed it had none.

Testing

MEASURE	VALUE
Test functions	463
Inline test lines (<code>#[cfg(test)]</code>)	9,028
Integration test lines (<code>tests/</code>)	6,450
Snapshot files	20
Line coverage	95.39%
Region coverage	96.56%

Three layers do distinct work. Unit tests cover module logic. Snapshot tests, via `insta`, pin the exact bytes of generated documents, so any change to output is visible in review rather than discovered by a consumer. Schema validation checks generated documents against the published CycloneDX and SPDX schemas, which is what caught the SPDX 3.0.1 timestamp precision problem described earlier.

Benchmarks are grouped as `parse`, `model` and `write` specifically so that a regression reports where it is rather than only that it happened.

Evaluation

We evaluated identity resolution on public workspaces. The corpus was assembled by querying GitHub code search for `pixi.lock`, retrieving one lockfile per distinct repository, and analyzing those that resolved for `linux-64`: 118 workspaces containing 15,722 packages, unfiltered for size, domain or quality.

Each workspace was processed twice, differing only in identification strategy, and both arms queried OSV on 30 September 2026.

	CONDA IDENTITY	PYPI IDENTITY	DIFFERENCE
Distinct advisories	2,158	3,432	+1,274
Critical	90	122	+32
High	1,043	1,643	+600
Medium	826	1,371	+545
Low	185	272	+87

Identity resolution surfaced **37.1 percent more advisories**, and lost none: across all 118 workspaces, zero advisories present under conda identification were absent under PyPI

identification.

Composition and reach:

- 14,120 of 15,722 packages (89.8 percent) were conda packages.
- 4,572 conda packages (32.4 percent) acquired a PyPI identity. The remaining 67.6 percent correctly acquired none.
- 96 of 118 workspaces carried at least one advisory when resolved.
- **45 of those 96 reported zero advisories unresolved.** Nearly half of the affected projects appeared entirely clean.

The newly visible findings concentrated in `pillow`, `tornado`, `mistune`, `jupyterlab` and `urllib3`, which is to say the ordinary load-bearing libraries of a scientific Python environment.

A negative result: both arms surfaced the same five KEV entries. Identity resolution added none. The KEV catalog skews toward network-reachable enterprise software and overlaps thinly with a scientific Python dependency set in either arm.

Document generation without enrichment is fast enough not to matter: a 129-package workspace parses and writes in under 20 milliseconds. Runs that query the network are dominated by the network, which is what `--timings` exists to show.

Related work

General-purpose scanners can identify Python distributions from an installed environment's metadata, sidestepping purl identity by reading `dist-info` directly. This works well on a materialized environment. It does not help when the artifact under analysis is a lockfile, which is the case in continuous integration and in supply chain review, and it does not describe a multi-platform lockfile's other platforms at all.

Conversion tools exist that transform one document format into another. They inherit whatever identity decisions the original generator made, so a document generated with unresolvable identifiers stays unresolvable through conversion.

The nearest adjacent work is the conda-forge PyPI name mapping itself, which pixi-sbom consumes rather than reimplements, and the purl specification, which is functioning as designed. The gap this tool fills sits between them.

Limitations

Mapping coverage bounds resolution. A conda-forge package absent from the mapping and from the lockfile's `purls:` field is not resolved. The tool will not guess, so coverage is exactly

the mapping's coverage.

Channels other than conda-forge. The mapping is conda-forge's. Packages from other channels resolve only through the lockfile's own record.

Reachability is out of scope. The tool reports that a vulnerable version is installed, not that the vulnerable code path is reachable. That distinction is what VEX expresses, and `--vex` emits a VEX document, but the assessment itself is human work.

Two recommendations the tool does not implement. It does not currently distinguish “no findings” from “could not query” in its output, and it does not report identity coverage as a first-class metric. Both would be valuable and neither is done.

Conclusion

Most of pixi-sbom's design is unremarkable, and deliberately so: a pipeline, an intermediate model, format writers behind it. The part that matters is the dual-identity scheme, and the reason it matters is not technical sophistication. It is that the obvious implementation, emitting the identifier of the ecosystem a package was installed from, produces documents that pass every validation check and answer no security question.

The evaluation puts a number on that. In 118 real workspaces, 37 percent of advisories were unreachable through conventional identifiers, and nearly half of the affected projects returned a clean report while carrying a median of 23 findings.

The fix is not clever. Carry both identities, take the queryable one from the ecosystem's own published mapping, refuse to guess, and discard nothing.

pixi-sbom is open source under Apache 2.0 at github.com/millsks/pixi-sbom. The corpus list, per-workspace measurements and analysis script accompany this paper.